Data is the cheapest efficiency lever

Why a better trillion beats a worse three, and the pipeline — sources, deduplication, filtering, mixing, decontamination — that turns a web crawl into training data you can trust

---

Here is the cheapest decision you will ever make in this book, and the one most teams make backwards. You have a budget for a 7B model: 2 trillion tokens of compute (Chapter 6 priced it at about 66K GPU-hours). A colleague has a 15-trillion-token web crawl on disk and proposes "just use more of it." Another has a 1.3-trillion-token subset that a classifier judged educational, and proposes training on that — three epochs of it, if necessary. Same compute, same model, same everything except which bytes go in. The second team wins, and not narrowly: on knowledge and reasoning benchmarks the educational subset beats the full crawl by margins that would take *several times* the compute to buy with architecture or scale[1]. Every chapter after this one fights for tens of percent of efficiency — faster attention, packed sequences, better parallelism. Data fights for multiples, and it does so with CPUs, before a single GPU is touched. That is why Part III opens here: the pipeline that turns a crawl into training data — sources and versioning, deduplication, quality filtering, mixing, a multilingual strategy, and the decontamination ring Chapter 9 demanded — is the highest-leverage engineering in the book, and it runs on the cheapest hardware you own.

10.1 Why a better 1T beats a worse 3T

The evidence arrived in three waves, and together they changed what "more data" means. The first wave was the open-dataset race of 2024. Hugging Face's FineWeb distilled 96 Common Crawl snapshots into 15 trillion tokens, then trained a classifier to keep only educational text and produced FineWeb-Edu: 1.3 trillion tokens, 92% of the data thrown away — and models trained on the small set were *dramatically* better on MMLU and ARC than models trained on the large one[1]. DataComp-LM made the same point with a different filter: a 7B model trained on their 2.6-trillion-token DCLM-Baseline reached 64% on MMLU, matching Llama 3 8B's quality with 6.6× less compute, and the single most important intervention in their ablations was model-based filtering[2]. NVIDIA's Nemotron-CC then showed the ceiling could rise further: an ensemble of classifiers plus synthetic rephrasing of marginal documents gave +5.6 MMLU over DCLM at equal tokens, while keeping four times more unique text for long runs[3].

The second wave answered the obvious objection — "but then I run out of tokens." Scaling-law work on data-constrained training found that repeating data is far cheaper than people feared: up to about four epochs over the same tokens, loss is nearly indistinguishable from training on fresh data; beyond that, each repetition buys less, decaying

toward nothing[4]. The fitted form is worth having in your head. If U is the number of *unique* tokens you own and R the number of times you repeat them, the effective data is

$$D_{\text{eff}} \;=\; U \;+\; U\,R^{\star}\left(1 - e^{-R/R^{\star}}\right), \qquad R^{\star} \approx 15,$$

where $R^{\star}$ is a fitted constant from the paper's experiments (an estimate; its value depends on the data). Read it as a budget: at $R = 4$, $D_{\text{eff}} \approx 4.5U$ — you got almost full value; at $R = 16$, $D_{\text{eff}} \approx 10.6U$ — the sixteenth pass of a document is worth a fraction of the first. The practical translation is the headline of this chapter: **a 1.3T-token curated set used for three epochs (≈3.9T effective) beats a 15T crawl used once**, and you still have compute left over.

The third wave is what the labs ship. Llama 3 pre-trained on roughly 15T tokens after a pipeline of URL, document, and line-level deduplication and two learned quality classifiers, ending in a deliberately engineered mix[5]; Qwen3 spent 36T tokens across 119 languages in three staged mixtures, the second of which raised the share of reasoning and synthetic data on purpose[6]. Nobody at the frontier "just uses the crawl." The rest of this chapter is the pipeline they run, at a scale you can afford.

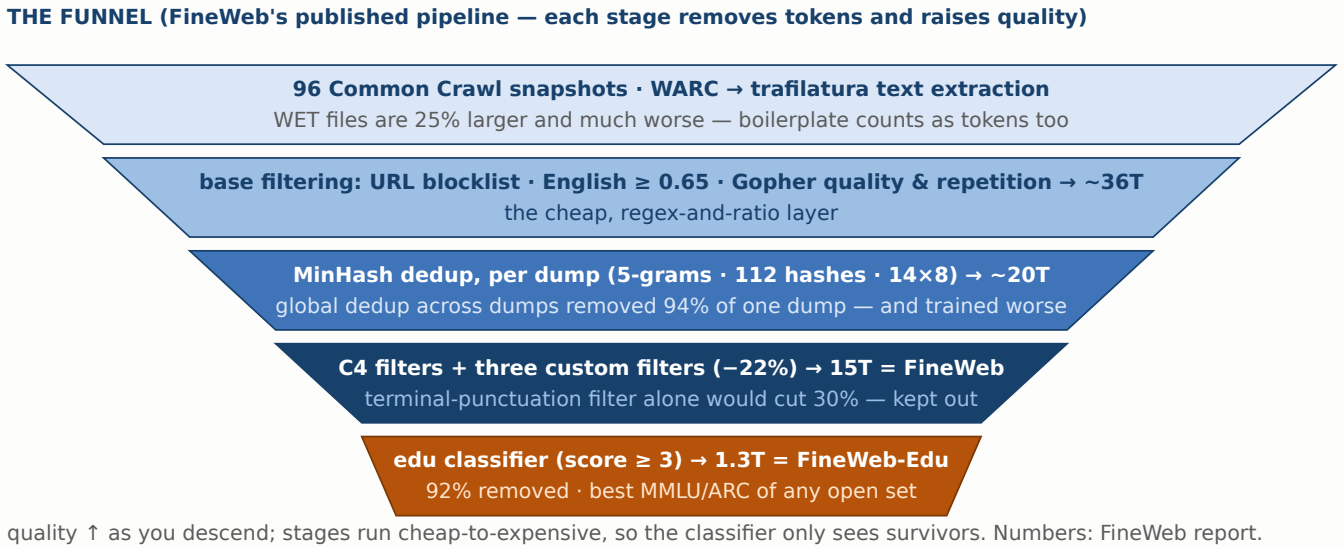


Figure 10.1 — The funnel. Every open frontier dataset is some version of this picture. Note the two counter-intuitive findings baked into it: more aggressive extraction (trafilatura over WET) and *less* aggressive deduplication (per dump, not global) both produced better models[1].

10.2 Sources and versioning: Hugging Face as the warehouse

You will not crawl the web. The open corpora of §10.1 — FineWeb and FineWeb-Edu, FineWeb2 for the non-English world, Dolma, DCLM, Nemotron-CC — already did the first 90% of the work and publish it on the Hugging Face Hub as versioned, streamable datasets, and the honest starting point for any team below frontier scale is to *start downstream of them* and spend your effort on what they don't do for you: your languages, your domains, your mixture, your decontamination. Three properties of the Hub matter operationally. **Streaming**: a 15T-token dataset is tens of terabytes; you never download it, you iterate shards. **Revisions**: every dataset is a git repository with commits, so "the data" is a commit hash, not a folder name — the unit of versioning Chapter 9 demanded for evaluations applies to training data

with the same force. **Cards**: the dataset card is where lineage lives — sources, filters, thresholds, license, known issues — and if you publish a derived dataset without one you have published a mystery.

```
# the warehouse, used honestly: stream, pin the revision, record the lineage
from datasets import load_dataset
src = load_dataset("HuggingFaceFW/fineweb-edu", name="sample-10BT",
                   split="train", streaming=True,
                   revision="v1.3.0")              # a commit, not "latest"
es  = load_dataset("HuggingFaceFW/fineweb-2", name="spa_Latn",
                   split="train", streaming=True)

LINEAGE = {"sources": [("HuggingFaceFW/fineweb-edu", "v1.3.0", "sample-10BT"),
                       ("HuggingFaceFW/fineweb-2", "main", "spa_Latn")],
           "filters": [], "dedup": {}, "decontam": {}}   # appended by each stage
for doc in src.take(3):
    print(doc["id"], doc["score"], doc["text"][:80])    # edu score ships with it
# on save: push_to_hub(..., commit_message=json.dumps(LINEAGE)) — the card
# is generated from LINEAGE; companion repo: data/pipeline/
```

Our running example for the rest of the chapter is exactly this pair: FineWeb-Edu's `sample-10BT` for English and FineWeb2's Spanish split for the language we actually serve. Both arrive deduplicated and filtered by their maintainers' thresholds — but *their* thresholds, tuned for *their* evaluations. The stages below are what you run on top, and each one appends to the lineage record.

10.3 Deduplication: exact, near, and how much is too much

Why dedup at all? Three reasons, in rising importance. Duplicates waste compute: §10.1's repetition law says the fourth copy of a document is worth less than the first, and web crawls contain documents thousands of times over. Duplicates distort: a boilerplate footer repeated across a million pages is a million gradient steps toward emitting that footer. And duplicates leak: the original study of this problem found that deduplicated models emitted memorized training text roughly an order of magnitude less often, and that near-duplicates also silently inflated evaluation by putting test-like text in training[7].

Exact deduplication is a hash: normalize whitespace and case, SHA-1 the document, drop repeats. It costs nothing and catches mirrors. **Near** deduplication is where the engineering lives, because two pages that differ by a date and a navigation bar are not byte-identical but are the same document. The tool is MinHash with locality-sensitive hashing, and it is worth understanding as arithmetic rather than library magic. Represent each document as its set of word 5-grams, S_A. The similarity you want is Jaccard, $J(A,B) = |S_A \cap S_B| / |S_A \cup S_B|$, but computing it for every pair is quadratic. MinHash's trick: apply a random hash to every 5-gram and keep the *minimum*; the probability two documents share the same minimum equals exactly their Jaccard similarity. So k independent hashes give k coin flips whose mean estimates J. LSH then avoids the pairwise comparison: split the k signatures into b bands of r rows, and call two documents candidates if *any* band matches exactly. The probability of becoming a candidate is

$$P(\text{candidate}) \;=\; 1 - \left(\,1 - J^{\,r}\,\right)^{\,b},$$

an S-curve whose steepness you tune with b and r. FineWeb uses $k = 112$ hashes as $b = 14$ bands of $r = 8$[1]; plug in and you reproduce their published thresholds exactly — at $J = 0.70$ the catch probability is 56%, at 0.75 it is 77%, at 0.80 it is 92%, at 0.85 it is 98.8%. That is what "we deduplicate documents ≥75% similar" actually means: a soft threshold set by two integers. (RefinedWeb used 9,000 hashes in 450 bands of 20 for a sharper curve, at 80× the cost; FineWeb found the cheap curve good enough.)

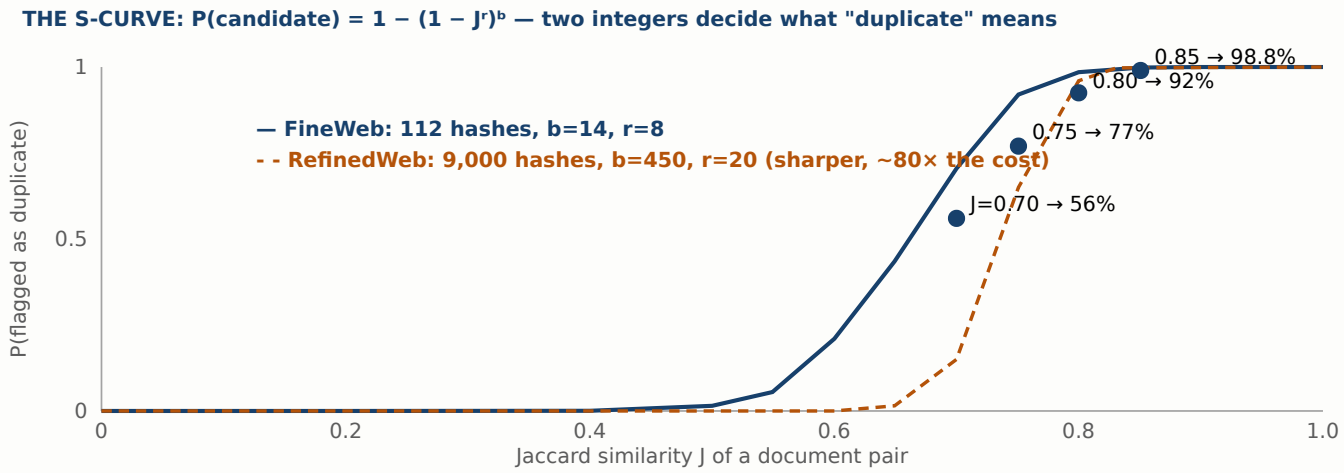


Figure 10.2 — What a dedup threshold really is. The soft knee near J ≈ 0.75 is set entirely by (b, r). A steeper curve catches fewer false positives below the knee and costs proportionally more hashes; FineWeb's ablations found the cheap curve lost nothing measurable[1].

```
# near-dedup in ~20 lines (datasketch); the companion repo scales it with
Spark/Ray
from datasketch import MinHash, MinHashLSH
import re, hashlib
def shingles(text, n=5):
    w = re.findall(r"\w+", text.lower())
    return {" ".join(w[i:i+n]) for i in range(max(1, len(w)-n+1))}
lsh = MinHashLSH(threshold=0.75, num_perm=112, params=(14, 8))   # b=14, r=8
seen_exact, kept = set(), []
for doc in stream:
    h = hashlib.sha1(" ".join(doc["text"].split()).encode()).hexdigest()
    if h in seen_exact: continue                          # exact dedup: free
    seen_exact.add(h)
    m = MinHash(num_perm=112)
    for s in shingles(doc["text"]): m.update(s.encode())
    if lsh.query(m): continue                             # near-duplicate of a kept
doc
    lsh.insert(doc["id"], m); kept.append(doc)
LINEAGE["dedup"] = {"exact": "sha1(normalized)", "near": "minhash 5-gram k=112
b=14 r=8",
                    "scope": "per source, per snapshot"}         # §10.3's lesson
```

Now the counter-intuitive part, which is the most useful finding in the FineWeb report. The natural instinct is to deduplicate *globally* — one index over all 96 snapshots. FineWeb tried it: the index removed 94% of the oldest dump (490B tokens down to 31B), and the *discarded* 94% trained a better model than the kept 6%[1]. The mechanism is selection: what survives global dedup against fifteen years of later crawls is precisely the content nobody ever re-posted — ads, boilerplate variants, low-value pages — while good content that got copied everywhere is removed as a "duplicate." Deduplicating each snapshot *independently* kept 20T tokens and matched the best open corpus. The lesson generalizes beyond Common Crawl: dedup is a filter, and like every filter it has a false-positive population; when the false positives are your best documents, less is more. Scope it per source and per snapshot, and ablate before going wider.

10.4 Quality filtering: heuristics, classifiers, and the last filter

Deduplication removes redundancy; filtering removes *junk*, and it comes in two tiers priced a thousand-fold apart. The cheap tier is heuristic: regexes and ratios that run at CPU speed over the whole crawl. Gopher's rules are the canonical set — drop documents under 50 or over 100K words, mean word length outside 3–10, more than 10% symbols, more than 90% bullet lines, fewer than 80% of words containing a letter, fewer than two stop words[8] — and C4's add line-level rules: drop lines that don't end in punctuation, JavaScript and cookie notices, "lorem ipsum," curly braces[9]. The craft is in *which* rules and *where* the thresholds sit, and FineWeb's ablations are the best public guide: three custom ratios (fraction of lines ending in punctuation ≤ 0.12, fraction of characters in duplicated lines ≥ 0.10, fraction of lines under 30 characters ≥ 0.67) removed a combined 22% of tokens and improved every benchmark, while C4's terminal-punctuation filter, applied naively, would have removed 30% of tokens for no gain[1]. The method behind those numbers is the one to copy: compute the statistic over the *kept* and *discarded* halves of a known-good dedup, look for distributions that differ, and set the threshold where they separate.

Table 10.1 — The heuristic layer, with what it costs. Removal fractions are FineWeb's published measurements on their crawl; yours will differ, which is the point of re-measuring.

FILTER	RULE	REMOVED	CATCHES
URL blocklist	adult / spam domain list	small	the obvious
language ID	fastText score ≥ 0.65 for the target language	large (non-target)	everything else; the knob for §10.6
Gopher quality	length, word length, symbol ratio, stop words	—	menus, code dumps, gibberish
Gopher repetition	duplicated-line / n-gram character fractions	—	boilerplate inside a document
C4 (minus terminal punct.)	JS/cookie lines, lorem ipsum, braces, length	—	template residue

FILTER	RULE	REMOVED	CATCHES
FineWeb custom ×3	punct-ended lines ≤ 0.12 · dup-line chars ≥ 0.10 · short lines ≥ 0.67	~22%	list pages, scrapers, navigation
C4 terminal punctuation	drop lines not ending in . ! ? "	~30%	*too much* — FineWeb left it out

The expensive tier is a **learned classifier**, and it is where the 2024 results came from. The recipe is now standard. Choose what "good" means with examples: FineWeb-Edu asked Llama-3-70B-Instruct to score 500K pages 0–5 for educational value, trained a small embedding model with a regression head on 450K of them (F1 0.82 on the held-out 45K), and kept score ≥ 3 — at a cost of 6,000 H100-hours to score 15T tokens[1]. DCLM did it cheaper and arguably better: a fastText classifier whose positives were instruction-style and explanatory text (OpenHermes, ELI5) and whose negatives were random crawl, keeping the top 10% by score[2]. Two design facts fall out. The classifier encodes a *taste* — educational, explanatory, reasoning-dense — and the taste is the single largest lever on downstream benchmarks you have; it is also a bias you are choosing on purpose (FineWeb-Edu is poorer at code and conversation precisely because its judge wasn't asked about them). And the threshold is a dial between quality and quantity: FineWeb-Edu at score ≥ 3 is 1.3T tokens; at ≥ 2 it is 5.4T[1], which is the one you want when §10.1's repetition budget says you need the tokens.

```
# the classifier tier, DCLM-style: cheap, explicit about its taste
import fasttext
clf = fasttext.train_supervised("quality_train.txt",     # __label__hq from
      lr=0.1, epoch=5, wordNgrams=2, dim=100)             #   OpenHermes/ELI5-like
text,
                                                          # __label__lq from
random crawl
def quality(doc):
    labels, probs = clf.predict(doc["text"].replace("\n", " ")[:4000])
    return probs[0] if labels[0] == "__label__hq" else 1 - probs[0]
scored = [(quality(d), d) for d in kept]
cut = sorted(s for s, _ in scored)[int(0.9 * len(scored))]   # keep the top 10%
kept = [d for s, d in scored if s >= cut]
LINEAGE["filters"] += [{"heuristic": "gopher+c4-minus-terminal+fineweb3"},
                       {"classifier": "fasttext hq/lq (OH+ELI5 vs crawl)", "keep":
"top10%"}]
# calibration you must run: a human reads 200 docs around `cut` — Ch. 11's
# judge/human agreement is the same discipline applied to the filter itself
```

The last filter: decontamination against your own benchmarks

Chapter 9 made the argument; this is where it runs. The final stage of the pipeline, after everything else has reduced the data to what you will actually train on, is to remove any document that overlaps the evaluation suite you intend to report. The standard mechanism is n-gram overlap in the GPT-3 lineage: for every benchmark item, extract its 13-grams; drop any training document that shares one[10]. It runs last for cost (fewest survivors to scan) and for honesty (it must see the *final* data, after any synthetic augmentation). It catches verbatim leakage only — paraphrased benchmark

items pass through, which is why Chapter 9 paired it with a private probe set and why Chapter 11 adds membership tests on the trained model. But verbatim leakage is the common case on a web crawl, and this stage is a few CPU-hours of insurance on a run that costs tens of thousands of GPU-hours.

```
# decontaminate LAST: the Ch. 9 suite's items become 13-gram tripwires
def ngrams(text, n=13):
    w = text.lower().split(); return {" ".join(w[i:i+n]) for i in range(len(w)-
n+1)}
trip = set()
for item in eval_suite_items():            # MMLU, GSM8K, ARC, HellaSwag, IFEval,
our probe
    trip |= ngrams(item["question"] + " " + item["answer"])
clean = [d for d in kept if not (ngrams(d["text"]) & trip)]
LINEAGE["decontam"] = {"method": "13-gram overlap", "suite": "table-9.1-v1",
                       "removed": len(kept) - len(clean)}
# log the removed documents: a spike in removals from one source is a leak you
# want to know about, not just filter
```

10.5 Mixtures and curriculum: what fraction of what, and when

After filtering you own several sources — English web, Spanish web, code, math, books, your domain documents — and the next decision is how much of each the model sees, which turns out to matter as much as the filtering did. The naive choice, proportional to size, lets the biggest source (web) dominate and under-trains everything the model will later be asked to do well. The frontier practice is an engineered mix: Llama 3 settled on roughly 50% general knowledge, 25% mathematics and reasoning, 17% code, and 8% multilingual data, tuned by scaling-law experiments on small models[5]. Read those numbers against the crawl: code and math are a few percent of the web, and the mix overweights them by an order of magnitude on purpose.

How to choose weights without a frontier lab's ablation budget? Two methods are worth knowing. **DoReMi** trains a small proxy model (280M parameters) with group distributionally-robust optimization — at each step it upweights the domains on which the proxy is doing *worst relative to a reference model* — and the resulting domain weights transfer to a 30× larger model, reaching the baseline's downstream accuracy with 2.6× fewer steps and gaining about 6.5 points on average across tasks[11]. In one line, the objective it solves is

$$\min_{\theta} \max_{\alpha \in \Delta} \sum_{i} \alpha_i \left[L_i(\theta) - L_i(\theta_{\text{ref}}) \right],$$

where α is the mixture over domains i, L_i the loss on domain i, and the subtraction of the reference loss stops the method from simply chasing the inherently hardest domain. **RegMix** is the pragmatic alternative: train dozens of tiny models on random mixtures, fit a regression from mixture weights to the loss you care about, and read the optimum off the regression[12]. Both say the same thing a different way: the mixture is a hyperparameter you can *fit*, cheaply, on models a hundred times smaller than the one you will train.

Curriculum is the mixture changing over time, and the current recipes converge on one shape. Qwen3's three stages are the clearest public example: over 30T tokens of general data at 4K context to build language and world knowledge; then about 5T tokens of higher-quality data with the share of STEM, code, reasoning, and synthetic text raised, under a faster-decaying learning rate; then hundreds of billions of tokens at 32K context to extend the window[6]. The logic is Chapter 20's annealing argument applied to data: the cheap, abundant tokens teach the base; the scarce, expensive tokens are spent where the learning rate is low and every example counts; the long-context tokens come last because they are the most expensive per token (Chapter 21). The operational rule: *spend your best data at the end*, and version each stage's mixture as its own dataset revision so the run is reproducible stage by stage.

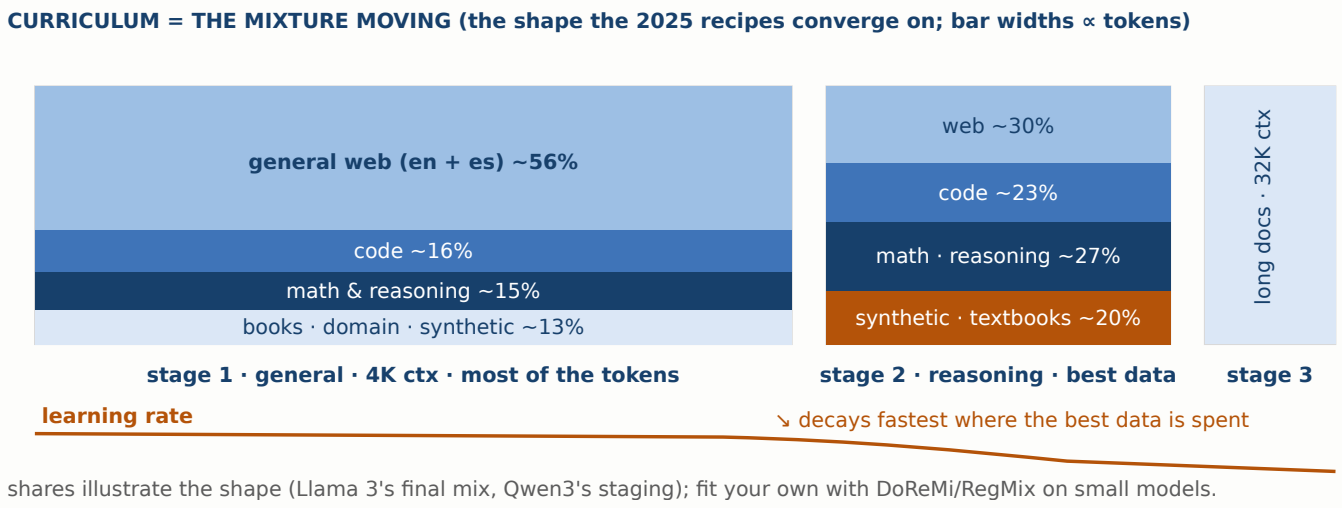


Figure 10.3 — Spend the best data last. The general stage builds the base from abundant tokens; the reasoning stage concentrates scarce, curated, and synthetic data under a decaying learning rate; the long-context stage is short because its tokens are the most expensive. Each stage is its own dataset revision.

10.6 Ordering the data: curriculum learning

Section 10.5 changed the mixture a few times over a run — three stages, each its own revision. Push that idea to its limit and you get a different question: if the model is going to see every document anyway, does the *order* of the documents matter? The default answer in every production recipe is a global shuffle, and the default is not stupid: SGD's convergence guarantees assume the batch is an unbiased sample of the distribution, and a shuffled stream is exactly that. But a shuffled stream also means that the model's first billion tokens include scientific papers it cannot yet parse, and its last billion tokens include children's stories it stopped learning from a trillion tokens ago. **Curriculum learning** is the bet that a schedule which roughly follows the model's competence — easy first, hard later — wastes fewer of those tokens. The evidence, which we will go through, says the bet pays at small scale and as a warm-up, and fades at a few hundred million parameters; the techniques for *scoring* difficulty, however, turn out to be useful regardless, because the same score that orders data can also prune it.

What "difficulty" means, in equations

Every curriculum needs a scalar per document, $d(x)$, and a schedule that says which range of d the model is allowed to see at training progress $t \in [0, 1]$. The scores in the literature fall into three families, cheapest first. **Surface statistics** cost nothing: sequence length, the fraction of unique words, and compression ratio — bytes after gzip over bytes before

— which measures how repetitive the text is. Readability formulas belong here too; Flesch Reading Ease is a linear function of syllables per word and words per sentence. **Linguistic scores** need a lexicon: the mean age of acquisition of the words (from psycholinguistic norms), their mean Zipf frequency, and verb variation, the number of distinct verbs over the square root of all verbs, which was the single score most correlated with a model's own per-sample loss in a six-scale study on Pythia models (Pearson 0.76)[13]. **Model-based scores** ask a language model how surprised it is. The canonical one is perplexity under a reference model θ_{ref}:

$$\text{PPL}_{\theta_{\text{ref}}}(x) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log p_{\theta_{\text{ref}}}(x_t \mid x_{<t})\right),$$

where T is the document's length in tokens and $p_{\theta_{\text{ref}}}(x_t \mid x_{<t})$ is the probability the reference assigns to the true next token (the quantity Chapter 2 built the whole loss around). Perplexity is the score that is most aligned with what the student model will actually find hard, because it *is* a model's loss, just a different model's — and it is also the only score in the list that is expensive, which is the engineering problem this section is mostly about.

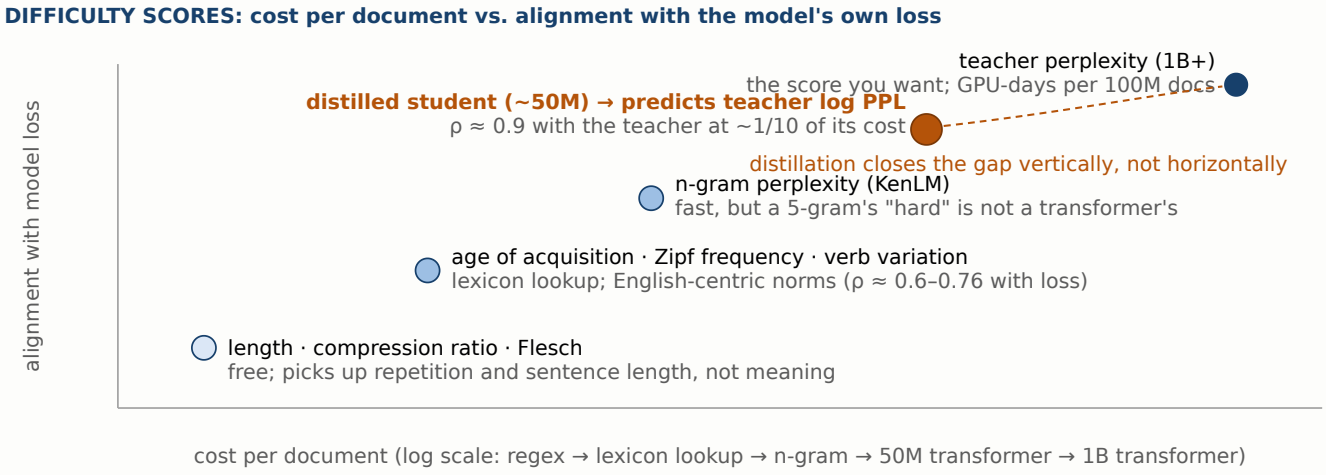


Figure 10.4 — Which difficulty score to pay for. Alignment is how well the score tracks the loss the student model will actually see. The free scores are enough for a warm-up curriculum; the model-based scores are what you need for pruning. The distilled student is the engineering trick that makes a teacher-quality score affordable at web scale.

Making a perplexity score affordable

Here is the arithmetic that rules out the obvious approach. Scoring a document with a 1B-parameter reference model costs a forward pass, about $2N = 2 \times 10^9$ FLOPs per token (Chapter 8). A 100-million-document corpus at roughly a thousand tokens per document is 10^{11} tokens, so 2×10^{20} FLOPs. At a realistic 40% of an H100's dense peak — call it 4×10^{14} FLOP/s — that is 5×10^5 seconds, close to six GPU-days for one pass over a corpus that is small by pre-training standards (these are estimates from the FLOP count, not measurements; real throughput for short-sequence inference is often worse). FineWeb has fifteen trillion tokens, two orders of magnitude more. The usual escape is an n-gram model — KenLM scores a document in microseconds on a CPU — but a 5-gram's idea of surprise is lexical: it finds rare words hard and long-range structure invisible, which is close to the opposite of what a transformer finds hard.

The fix is distillation, applied not to generation but to the score itself. Train a small student f_ϕ — a transformer of a few tens of millions of parameters, multilingual if your corpus is — to *regress* the teacher's log-perplexity:

$$L(\phi) = E_{x \sim D}\Big[\ \big(\ f_\phi(x) - \log\mathrm{PPL}_{\theta_{\text{ref}}}(x)\ \big)^2\Big],$$

and judge it not by the regression loss but by the rank correlation (Spearman ρ) between its scores and the teacher's on held-out documents, because a curriculum only ever uses the *order*. Two things make this work better than it sounds. The target is a single scalar per document, so the student does not need the teacher's capacity to model language — only enough to recognize what makes text hard, which is a far easier function. And the training set for the student is cheap: a few hundred thousand teacher-scored documents, a GPU-hour or two, after which the student scores the remaining corpus at roughly the ratio of the parameter counts — a 50M student is about twenty times fewer FLOPs per token than a 1B teacher, and in practice around an order of magnitude more documents per second once memory and batching are accounted for. A student in this regime reaching $\rho \approx 0.9$ with its teacher brings the six GPU-days above down to well under one, and makes teacher-quality scoring of a full crawl a budget line rather than a research project. The same recipe applies to any expensive scalar you want at scale, which is why the educational classifier of §10.4 is itself a distilled score: Llama-3-70B labels on 450k documents, a small regressor on everything else[1].

```
# difficulty = log PPL under a small reference model (companion repo: distillation
step)
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
tok = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM2-135M")
ref = AutoModelForCausalLM.from_pretrained("HuggingFaceTB/SmolLM2-135M",

torch_dtype=torch.bfloat16).cuda().eval()
@torch.no_grad()
def log_ppl(texts, max_len=1024):
    b = tok(texts, return_tensors="pt", padding=True, truncation=True,
            max_length=max_len).to("cuda")
    logits = ref(**b).logits[:, :-1]                      # predict token t+1
from prefix
    tgt, mask = b.input_ids[:, 1:], b.attention_mask[:, 1:]
    nll = torch.nn.functional.cross_entropy(logits.transpose(1, 2), tgt,
reduction="none")
    return ((nll * mask).sum(1) / mask.sum(1)).float().cpu()   # mean NLL per doc
= log PPL
for doc in stream:
    doc["difficulty"] = float(log_ppl([doc["text"]])[0])     # one scalar per
document
# then: bin by quantile of d(x) and feed bins by a pacing function g(t)   (next
section)
```

Schedules: how to spend the order you bought

With a score in hand, the schedule decides what the model sees when. Write $g(t) \in [0, 1]$ for the fraction of the corpus, sorted easy-to-hard, that is *unlocked* at training progress t; batches are drawn uniformly from the unlocked part. The variants in the literature are mostly choices of g[14]. A **vanilla curriculum** is $g(t) = t$ with no re-sampling: the model walks the sorted corpus once, which maximizes the ordering effect and also maximizes the damage, because by the end

the batches contain only hard documents and the gradient noise climbs. **Pacing functions** keep the unlocked set growing but draw from all of it, so easy data is never fully gone; a root pacing $g(t) = \sqrt{t}$ unlocks fast and spends most of training on a near-uniform mix, a linear one is in between, and a step schedule unlocks one bin at a time. **Interleaving** cycles through bins so that each unit of training contains every difficulty level in a fixed ratio, which is just a mixture with difficulty as the domain variable. And the **warm-up curriculum** applies any of these for the first fraction of training and then switches to a plain shuffle — the variant that, in a 200-model study on up to 100B tokens, gave the most durable gains: up to 3.5% on an eight-benchmark average, and 18–45% fewer steps to reach the baseline's score, with compression ratio, lexical diversity, and readability as the scores that worked best[14].

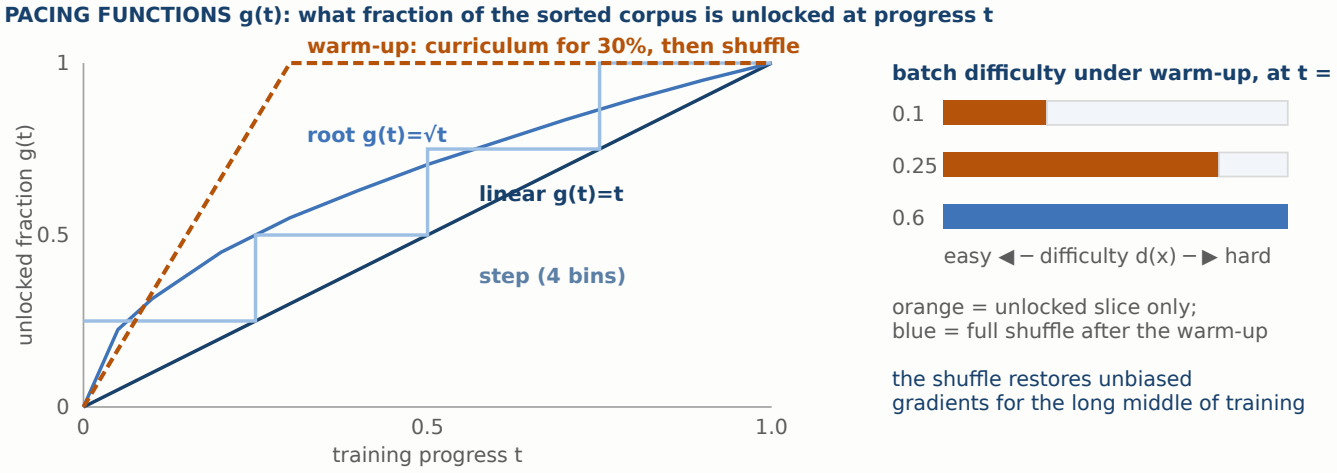


Figure 10.5 — Four ways to spend an ordering. Vanilla and step schedules end on hard-only batches; root pacing is nearly a shuffle after the first fifth; the warm-up schedule uses the curriculum where it helps most — the first steps, when the model can only learn easy structure anyway — and then stops biasing the batches.

Why would the warm-up form be the durable one? Because of what a curriculum actually does to optimization. The six-scale Pythia study found that all orderings pass through the same latent training phases; a curriculum does not create new ones, it changes what the model sees inside each phase, and its measurable effect at small scale is a *lower gradient noise scale* — batches of similar difficulty give more consistent gradients — along with fewer of the late-training pathologies associated with saturating the output softmax[13]. Both effects are strongest when the model is small relative to the data, which is why the same study saw the best ordering vary by scale (frequency at 14M, verb variation at 31–70M, age of acquisition at 160M) and saw random ordering win outright at 410M and 1B[13]. For a production-scale model, then, the honest expectation is: a short easy-first warm-up may buy a few percent of early compute, and the data-stage curriculum of §10.5 — which is a curriculum in *quality* and *domain* rather than in difficulty — is where the real gains live.

The score is worth more than the order: perplexity pruning

The scoring machinery outlives the curriculum. The same reference-model perplexity, used to *select* rather than to sort, is one of the better-tested pruning methods: with a 125M-parameter reference model scoring the corpus and a 3B model trained on the kept subset, keeping the *middle* of the perplexity distribution (not the lowest — the easiest documents are boilerplate, and the hardest are noise) improved average downstream accuracy by up to 2.04 points and reached the unpruned baseline with up to 1.45× fewer steps, with the gains surviving in the over-trained and data-constrained regimes that matter in practice[15]. The rule of thumb that falls out is a two-sided cut on $\log\mathrm{PPL}$: drop the bottom few

percent (near-duplicate templates and lists the model already knows) and the top tail (encoding errors, tables, wrong-language fragments the language filter missed), then train on what is left in whatever order your stage schedule prescribes. Note what the reference model's size tells you: 125M is enough to see which documents are informative for a model twenty-four times larger, the same lesson as the distilled student above and the 280M DoReMi proxy of §10.5 — the shape of "what is good data" is visible to small models, and the expensive model should only ever be trained on it.

> **WHAT TO ACTUALLY DO**
>
> Score every document once, with a distilled perplexity model if your corpus is above a few billion tokens and with a 100–150M reference model otherwise, and store the score in the lineage record next to the quality-classifier score. Use it twice: as a two-sided prune (drop the bottom and top few percent after inspecting samples from each tail) and, if you want the curriculum, as a warm-up ordering for the first 10–30% of the general stage, followed by a plain shuffle. Keep the §10.5 staged mixture as the real curriculum. Re-measure on your own small models before promising a number: the published gains are strongly scale-dependent.

10.7 Multilingual strategy: the Latin American corpus

Here is the number that shapes everything a Spanish-first model does: in the Common Crawl snapshots the 2025 benchmark work analyzed, English is about 43% of the text and Spanish about 4.6%[16]. Portuguese is smaller still, and the Latin American varieties of both are a fraction of those fractions. A pipeline that "just filters the crawl" therefore produces a model whose Spanish competence is an afterthought of its English competence — and the interpretability literature says that is literally what happens inside: the middle layers of English-dominant models reason in an English-shaped space and translate at the end, with measurable accuracy costs for low-resource languages (the experiment we ran on this is Phase 2's first lab). The fix is not a flag; it is three pipeline decisions.

Source the language on purpose. FineWeb2 is the starting point: one pipeline adapted to over 1,000 languages from almost 100 crawl snapshots, 20 TB and 5 billion documents, with per-language filtering thresholds and a *rebalancing* step that weighs duplication count and quality together — the finding being that in non-English data, documents that appear several times are often the valuable ones, and upweighting them helps[17]. Take its Spanish and Portuguese splits, then add what the web under-represents: government and legal corpora, regional news archives, textbooks, and your own domain documents, each with its license checked and recorded in the card. **Filter in the language.** Every threshold in §10.4 was tuned on English; stop-word lists, mean word length, and the educational classifier's taste all transfer badly. Re-derive the heuristic thresholds from your Spanish distribution the FineWeb way, and train the quality classifier on Spanish positives (annotated by a strong model *in Spanish*) rather than translating the English one. **Mix above the crawl's share.** If the target is a model that serves Spanish, the mixture must overweight it the way Llama 3 overweights code: a Spanish share of 20–35% of general-web tokens is where regional efforts land, far above the crawl's 4.6%, with English retained because it carries the bulk of the world's technical and scientific text and because cross-lingual transfer from it is real (Chapter 11 measures it). The tokenizer tax of Chapter 12 — Spanish costing up to 1.5× the tokens of English for the same content under an English-tuned vocabulary — means the *token* share must be computed after tokenization, not before, or your 30% is really 20%.

10.8 What the big labs do

Table 10.2 — Pre-training data, as shipped. Token counts are from the cited reports; the mixes are what they disclose.

MODEL / DATASET	TOKENS	PIPELINE HIGHLIGHTS	THE LESSON
FineWeb / FineWeb-Edu[1]	15T / 1.3T	trafilatura; per-dump MinHash (14×8); Gopher + C4 + 3 custom filters; LLM-labeled edu classifier	The published ablations are the field's textbook; copy the method, re-measure the thresholds
DCLM-Baseline[2]	2.6T (from a 240T pool)	fastText quality classifier (OH-2.5 + ELI5 positives), top 10%	Model-based filtering is the single largest lever; a cheap classifier suffices
Nemotron-CC[3]	6.3T	classifier ensemble; synthetic rephrasing of marginal docs; fewer heuristics	Quality *and* quantity: rephrase instead of discard when you need the tokens
Dolma / OLMo 2[18]	3T open corpus; OLMo 2 ~4–5T	fully open sources, toolkit, and mixes; staged "Dolmino" high-quality mix for annealing	Everything can be published — and the annealing mix is where curated data goes
Llama 3[5]	~15T	URL + document + line-level dedup; fastText and RoBERTa quality classifiers; 50/25/17/8 mix	The mix is engineered by scaling-law experiments, not inherited from the crawl
Qwen3[6]	36T, 119 languages	three stages (general → reasoning → long context); trillions of synthetic tokens from Qwen2.5 specialists	Curriculum and synthetic data are now core, not garnish
FineWeb2[17]	20 TB, 1,000+ languages	per-language thresholds; duplication-aware rebalancing	Non-English needs its own thresholds — and its duplicates may be signal

10.9 Practical considerations

The pipeline is a CPU job; make it a fast one. Processing 15T tokens through heuristics and MinHash is a few thousand CPU-core-hours on a Spark or Ray cluster — cheap next to training, but slow enough to block you if it is a single-node Python script. The order of stages is a cost argument: run the cheapest, highest-removal filters first (language ID, URL, Gopher) so the expensive ones (MinHash, classifier, decontamination) only see survivors. Shard everything by source and snapshot so stages are embarrassingly parallel and so a bug in one source doesn't force a

global rerun. And checkpoint the pipeline like a training run: each stage writes an immutable output with its lineage, so stage 4 can be redone without redoing stages 1–3.

Measure the filter, not just the model. Every filter has a false-positive population, and §10.3's global-dedup story is what happens when you don't look at it. Before training, sample 200 documents from the *discarded* side of each stage and read them; if more than a handful are documents you would want, the threshold is wrong. After training, Chapter 9's sliced held-out losses — one per source and language — will tell you within a few thousand steps whether a mixture change helped or quietly traded Spanish for code.

Licenses, PII, and provenance are pipeline stages, not legal footnotes. Record the license of every source in the card; run a PII pass (emails, phone numbers, government ID patterns) with redaction before dedup, since redaction changes hashes; keep the URL and crawl date for every surviving document. When a takedown or an audit arrives — and in a regulated deployment it will — provenance is what lets you answer in hours instead of retraining.

Common pitfalls

Global dedup across years of crawl.

Removes your best, most-copied content and keeps the junk nobody repeated. Dedup per source and per snapshot; ablate before widening the scope.

English thresholds on Spanish data.

Stop-word counts, word-length bands, and the edu classifier's taste all shift by language. Re-derive from the target-language distribution.

Mixing by bytes instead of tokens.

Under an English-tuned tokenizer, Spanish and code inflate; a 30% byte share can be a 20% token share. Compute the mix after tokenization (Chapter 12).

Decontaminating before augmentation.

Synthetic data generated after the decontamination pass can reintroduce benchmark-shaped text. Decontaminate last, and again after Chapter 27's factory runs.

A classifier with a secret taste.

If you can't state in one sentence what the quality classifier rewards, you can't predict what the model will be bad at. Write the sentence into the card.

"Latest" as a dataset version.

A Hub dataset moves under you between runs. Pin the revision hash in the lineage record, every source, every time.

10.10 Decision framework

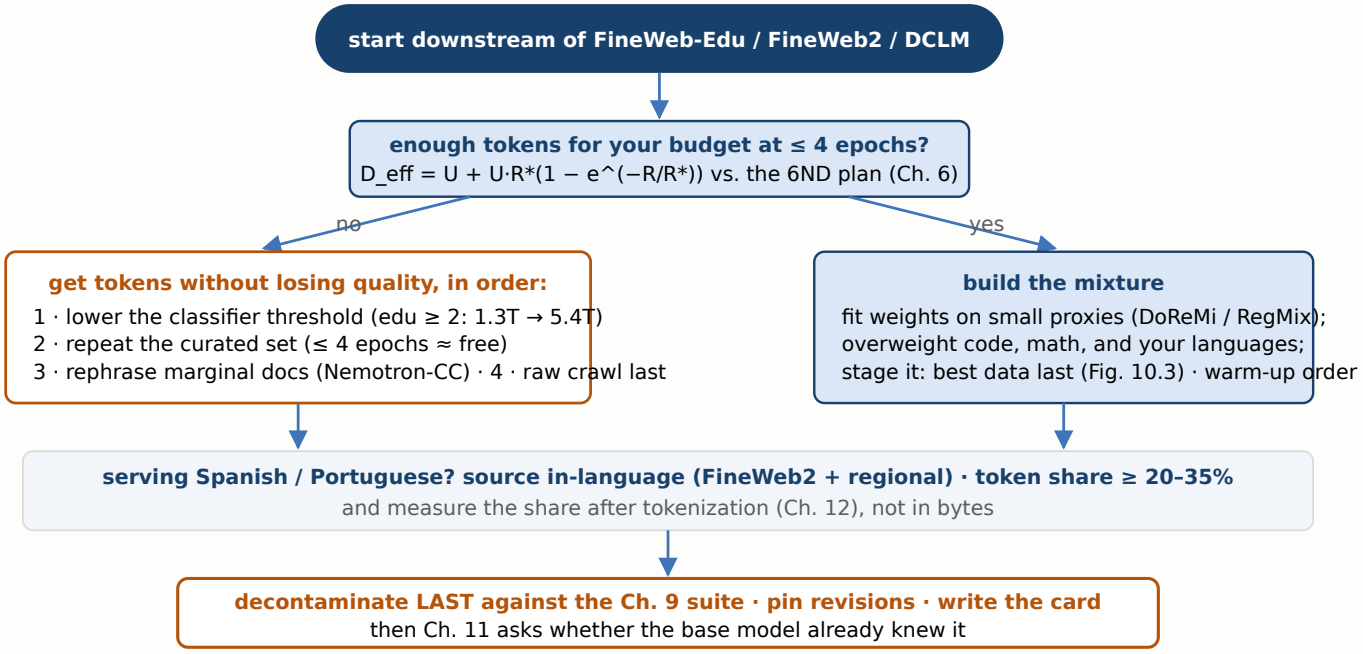


Figure 10.6 — Data decisions in order. Start from the open curated corpora, close the token gap with the cheap moves before the expensive ones, fit the mixture on small models, treat your languages as first-class, and finish with the decontamination ring. Every arrow is a dataset revision.

Cost-benefit, bluntly. The whole pipeline is CPU work costing a few percent of the training run, and its payoff is measured in multiples of compute: DCLM-Baseline matched Llama 3 8B's MMLU with 6.6× less compute by filtering alone[2]. Starting downstream of the open corpora captures most of that for a week of engineering; the quality classifier for your own taste and language is the next week and the next multiple; fitting the mixture on proxies is a few hundred small-model GPU-hours that pay back on the first real run. **How this scales:** at single-node scale, data is the *only* lever that moves quality by multiples, and the §10.1 repetition law means a small, curated, repeated set is the right design; at Chapter 14's scale the pipeline becomes a cluster job and the mixture becomes a scaling-law study; at Chapter 29's scale the data team is the largest team, which is the honest summary of how the frontier labs are organized.

10.11 Summary

Data is where efficiency compounds. A curated trillion beats a raw three — the open results of 2024 (FineWeb-Edu, DCLM, Nemotron-CC) established it with ablations, and the data-constrained scaling law says you can repeat the curated set up to about four times before the advantage fades. Order matters less than quality: a difficulty-scored warm-up curriculum buys a little at small scale, while the same perplexity score, used to prune the tails, pays at every scale. The pipeline that produces such a set runs on CPUs, in a fixed order: start downstream of the open corpora on the Hub, pinned by revision and documented by card; deduplicate exactly and then approximately with MinHash-LSH, whose threshold is two integers in an S-curve, and scope it per source and per snapshot because global deduplication removes your best content; filter with cheap heuristics whose thresholds you re-measure on your own distribution, then with a learned classifier whose taste you can state in one sentence; fit the mixture on small proxy models and stage it so the

best data is spent under the lowest learning rate; treat Spanish and Portuguese as first-class — sourced from FineWeb2 and regional corpora, filtered in-language, overweighted far above their 4.6% share of the crawl, and measured in tokens, not bytes; and decontaminate last, against the exact suite Chapter 9 froze, so that every number you later report is one you can defend. What this chapter cannot tell you is whether the model you are about to train has already seen any of it — that question, and the judge-and-human validation of what you built, is Chapter 11.

References

1. Penedo, G., et al. (2024). The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. arXiv:2406.17557; and the accompanying Hugging Face report. 15T tokens from 96 snapshots; per-dump MinHash (5-grams, 112 hashes, 14×8) with the global-dedup ablation; custom filters removing ~22%; FineWeb-Edu classifier (Llama-3-70B-Instruct labels, threshold 3, F1 0.82) yielding 1.3T tokens (5.4T at threshold 2).
2. Li, J., et al. (2024). DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. arXiv:2406.11794. 240T-token pool; DCLM-Baseline 2.6T; 7B model at 64% MMLU, matching Llama 3 8B with 6.6× less compute; fastText quality classifier.
3. Su, D., et al. (2024). Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset. arXiv:2412.02595. 6.3T tokens; classifier ensembling and synthetic rephrasing; +5.6 MMLU over DCLM at 1T tokens on 8B models.
4. Muennighoff, N., et al. (2023). Scaling Data-Constrained Language Models. *NeurIPS*. arXiv:2305.16264. Up to ~4 epochs of repetition costs almost nothing; the effective-data form with fitted R*.
5. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. ~15T tokens; URL/document/line-level dedup; fastText and RoBERTa quality classifiers; the 50/25/17/8 mix.
6. Qwen Team (2025). Qwen3 Technical Report. arXiv:2505.09388. 36T tokens, 119 languages; three-stage pre-training; trillions of synthetic tokens from Qwen2.5 specialists.
7. Lee, K., et al. (2022). Deduplicating Training Data Makes Language Models Better. *ACL*. arXiv:2107.06499. Near-duplicate removal; memorized output emitted ~10× less often; train–test overlap in standard corpora.
8. Rae, J., et al. (2021). Scaling Language Models: Methods, Analysis & Insights from Training Gopher. arXiv:2112.11446. The MassiveText quality and repetition heuristics.
9. Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5 / C4). *JMLR*. arXiv:1910.10683. The C4 line-level filters.
10. Brown, T., et al. (2020). Language Models are Few-Shot Learners. *NeurIPS*. arXiv:2005.14165. 13-gram overlap decontamination of training data against benchmarks.
11. Xie, S. M., et al. (2023). DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining. *NeurIPS*. arXiv:2305.10429. 280M proxy with group DRO; 2.6× fewer steps to baseline accuracy; +6.5 points on The Pile.
12. Liu, Q., et al. (2024). RegMix: Data Mixture as Regression for Language Model Pre-training. arXiv:2407.01492. Fit mixture→loss regression on many small runs; read off the optimum.

13. Elgaar, M., & Amiri, H. (2026). Curriculum Learning for LLM Pretraining: An Analysis of Learning Dynamics. arXiv:2601.21698. Pythia 14M–1B on 300B tokens; age-of-acquisition, word-frequency, and verb-variation orderings; gains at ≤160M, random best at 410M–1B; lower gradient noise scale under curricula.

14. Zhang, Y., Mohamed, A., Abdine, H., Shang, G., & Vazirgiannis, M. (2025). Beyond Random Sampling: Efficient Language Model Pretraining via Curriculum Learning. arXiv:2506.11300. 200+ models on up to 100B tokens; vanilla, pacing, interleaved, and warm-up curricula; compression ratio, MTLD, and Flesch as the best scores; up to 3.5% sustained gains and 18–45% fewer steps to baseline.

15. Ankner, Z., et al. (2024). Perplexed by Perplexity: Perplexity-Based Data Pruning with Small Reference Models. arXiv:2405.20541. 125M reference model pruning for a 3B model; +2.04 average downstream points; baseline reached with 1.45× fewer steps; keep the middle of the perplexity distribution.

16. Singh, S., et al. (2024). Global MMLU: Understanding and Addressing Cultural and Linguistic Biases in Multilingual Evaluation. arXiv:2412.03304. Common Crawl language shares: English ≈ 43.4%, Spanish ≈ 4.6%.

17. Penedo, G., et al. (2025). FineWeb2: One Pipeline to Scale Them All — Adapting Pre-Training Data Processing to Every Language. arXiv:2506.20920. 1,000+ languages, 20 TB, 5B documents; per-language adaptation; duplication-aware rebalancing.

18. Soldaini, L., et al. (2024). Dolma: An Open Corpus of Three Trillion Tokens. *ACL*. arXiv:2402.00159; and OLMo Team (2025). 2 OLMo 2 Furious. arXiv:2501.00656. Fully open corpus and toolkit; the staged high-quality annealing mix.